

ハーネスエンジニアリング入門

モデル単体ではできないことを、周辺で動かす設計

AI社会実装勉強会 第58回 | 2026/04/30

西岡 賢一郎

DataInformed CEO / D-stats CTO

Claude に毎回同じ注意を言っていないませんか？

「commit 前に lint 走らせて」

「rm -rf 絶対やめて」

「このチケット見てから着手して」

→ 今日 30 分は『毎回言わなくていい』を作る話

\$9

20 分

→ 動かないプロトタイプ

バグだらけ・遊べない・見栄えも悪い

\$200

6 時間

→ ちゃんと遊べるゲーム

20 倍以上のコスト・20 倍の時間で、プロトタイプ止まりが遊べるゲームになる

(原因: Planner / Generator / Evaluator の 3 エージェント構成。"ハーネス" の効果ではなく"マルチエージェント設計" の効果)

この差はどこから来たのか？

ハーネス(harness) = "Claude を呼ぶループ + ツール呼び出しを実インフラに繋ぐ仕組み"

"the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure" — Managed Agents, 2026/4

本質: モデル単独ではできないことの "仮定" を符号化したもの

"every component in a harness encodes an assumption about what the model can't do on its own" — Harness design, 2026/3

補助イメージ(※公式語源解説ではない): 馬具の harness のように、モデルを支え暴走させずに走らせる周辺装備

4b / ハーネスは多様 — \$9 vs \$200 もハーネスの差

\$9

単一エージェント
ハーネスなし
動かないプロトタイプ

\$200

3 エージェントを束ねるハーネス
Planner / Generator / Evaluator
Playwright MCP で実機検証 → 遊べるゲーム

3 エージェント協調はハーネスの代表例。今日扱う Claude Code hooks は同じ "外側で支える" 思想の最小実装(発表者の整理)

用語注: Anthropic 公式表記は "harness design"。「ハーネスエンジニアリング」は本発表の便宜的呼称

5 / 今日の地図 — AI実装手法の進化アーク



※ この『3 段 + 1 メタ層』整理は発表者のもの。Anthropic 自身は段階進化として打ち出していない。Managed Agents は公式に "meta-harness" と定義され、Prompt/Context/Harness と同列の engineering 手法ではない

ひとことで言うと

どう書けば期待通り動くか

Few-shot

Chain of Thought

Role prompting

Zero-shot CoT

単発質問の最適化 / 会話を進めるドライバーは『人』

みなさんも自然と身につけてきたフェーズ

ひとことで言うと

『何を入れて、何を入れないか』を設計する

「コンテキストは、限界効用が逓減する有限資源として扱わねばならない」

"Context, therefore, must be treated as a finite resource with diminishing marginal returns." — Anthropic, 2025/9

コンパクション

長い対話を要約して圧縮する技法

context rot

文脈が長くなるほど想起精度が下がる

サブエージェント

別文脈に切り出して並列処理させる

プロンプトを書くから、文脈の置き場を設計する、への視点転換

ひとことで言うと

セッションが切れても、仕事を継続できる仕組み

「シフト制で働くエンジニアたち。新人が来るたびに、前のシフトで何が起きたかの記憶はゼロ」

"...where each new engineer arrives with no memory of what happened on the previous shift" — Anthropic, 2025/11

★ ハーネスがない素の Claude が陥る 3 失敗

① 一発書き

アプリ全体を一発で書こうとして
コンテキスト切れで止まる

② E2E 未確認

書き換えはするが
end-to-end で動くか確認しない

③ 早期完了申告

テストを 1 つ通したら
「完了です」と申告

Anthropic 公式が 2025 年から使い始めた言葉。業界横断の定着はこれから

★ Prompt/Context/Harness と並ぶ engineering 手法ではない。
ハーネスを載せるホステッドサービス(meta-harness)

"Managed Agents is a meta-harness ... unopinionated about the specific harness that Claude will need in the future."

公式トライアド: 3 つの独立成分

harness

Claude を呼ぶループ

sandbox

実行環境

session

追記専用ログ

p95 TTFT 90%+ 削減

\$0.08 / セッション 1 時間

「ハーネスは "モデルが単独でできないことの仮定 " を符号化したもの。モデルが進化するとその仮定は陳腐化する」

"Harnesses encode assumptions that go stale as models improve."

Managed Agents は "あなたが書いたハーネスを載せる土台"。だから今日学ぶ価値があるのは、まずその "載せる中身" を 1 行から書ける hooks

プロンプトは "お願い"、hooks は "ルール"

28

イベント

5

ハンドラ種

2

今日触るのは
(PreToolUse / SessionStart)

ハンドラ 5 種: `command` / `http` / `mcp_tool` / `prompt` / `agent`

hooks は Anthropic 公式記事 4 本のいずれにも "harness" として名指しされていない。本日は『ハーネスの本質 = モデル単独ではできないことを外側で補う』に最も近い最小実装として hooks を選んだ(発表者の整理)

用語注: MCP = Claude が外部ツール(DB / Slack / GitHub 等)を呼ぶための共通プロトコル。Anthropic 公式機能

10-bis / hooks 採用前の 4 階層 — 個人 → チーム → 組織

`~/.claude/settings.json` = 個人専用

`.claude/settings.json` = チーム共有 (git に commit)

`.claude/settings.local.json` = ローカル上書き (.gitignore 推奨)

`managed policy` = 組織で強制

個人で試す → チームに撒く → 組織で固める

困ったら 2 つ覚える: `/hooks` メニュー / `claude --debug`

事業視点: = 個人実験から組織標準へ昇格させる経路が組み込まれている

※ 今回は settings.json の permission の話は割愛

11 / PreToolUse で危険コマンドを止める

Before (フックなし)

rm -rf → そのまま実行

After (hook あり)

rm -rf → exit 2 → 止まる

settings.json

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.command // \\\"\\\"' | grep -qE 'rm -rf|DROP TABLE' && exit 2 || exit 0"
      }]
    }]
  }
}
```

matcher = Bash ツール呼び出しだけに反応

jq = 標準入力の JSON を読む

exit 2 = ブロック発動の合図

事業視点: = 本番事故の事前防止

12 / 動作イメージ — rm -rf がブロックされる様子

```
$ cd ~/demo-harness && claude  
現在のブランチ: demo/harness-talk  
未コミット変更: 1 件
```

```
> ~/no-harness/foo を rm -rf で消して  
Bash: rm -rf ~/no-harness/foo  
✗ Blocked: dangerous command detected by harness hook  
→ Claude: フックでブロックされました。別の方法を試しますか?
```

12-bis / もう一つの実用例 — PostToolUse で lint 自動実行

hooks は "止める" だけじゃない。Edit/Write の "後" に決まった処理を流せる

```
{ "hooks": { "PostToolUse": [{  
  "matcher": "Edit|Write",  
  "hooks": [{  
    "type": "command",  
    "command": "jq -r '.tool_input.file_path' | xargs -r npx eslint --fix"  
  }]  
}] }
```

matcher "Edit|Write" = Claude がファイルを書き換えた直後に発火

tool_input.file_path = Claude が編集したファイルのパスがJSON で渡る

eslint --fix = そのファイルだけをlinter で整形 (prettier / gofmt / black なんでも)

事業視点 := AI が書いたコードを、レビューに出す前に「自社規約」に揃える / 毎日の手間が減る

重い: autonomous-coding

Python 約 700 行 + Markdown 約 1000 行

200+ feature を無人実装

数十時間・数百ドル

モード: 無人で動く

軽い: Claude Code hooks

settings.json 数行

rm -rf を 1 個防ぐ

0 円・数秒

モード: 人と並走

共通項: モデル単独ではできないことを "周辺で補う"

規模の差は予算の差。思想は同じ。—— まずは右端から。

★ ただし、その思想に賞味期限はあるか ? — 原則④で答え合わせ

Claude が起動した瞬間に、いま何が起きているかを自動で渡しておく仕組み

CLAUDE.md

静的

プロジェクトの説明
コーディング規約
毎回変わらない情報

SessionStart hook

動的

現在のブランチ
未コミット数
起動のたびに最新の状態を流し込む

事業視点: = ナレッジを毎回最新で注入。社内運用情報・担当チケットを自動文脈化

15 / 動作イメージ — claude 起動だけで git 状態が文脈に入る

```
$ claude
```

```
[SessionStart hook]
```

```
現在のブランチ: demo/harness-talk
```

```
未コミット変更: 1 件
```

```
最終コミット: a1b2c3d initial
```

> 私の今のブランチ名と未コミット変更の数を教えて、git は打たないで。

→ **demo/harness-talk ブランチで、未コミット 1 件です。**

settings.json 抜粋 (SessionStart の plain stdout 方式)

```
{ "hooks": { "SessionStart": [{ "hooks": [{  
  "type": "command",  
  "command": "echo \"現在のブランチ: $(git branch --show-current)\\n未コミット変更: $(git status -s | wc -l) 件\\n\""  
}] } ] }
```

matcher 不要 / 4 ソース発火 (startup・resume・clear・compact) / 応用: gh issue view、社内 API、Jira MCP

Session 1 — Initializer

プロジェクト初期化 / 200+ feature を JSON 化 / init.sh + claude-progress.txt 生成 / 最初の git commit

↓ 各セッション *fresh context* で復帰

Session 2 以降 — Coding agent (毎セッション)

feature_list.json / claude-progress.txt / git log を読んで復帰 → 機能を 1 個ずつ実装 → commit

→ スライド 8 の『シフト制エンジニア』の比喩がそのまま実装になっている

agent.py 約 206 / client.py 約 122 / security.py 約 359 / prompts 計 約 1000 行 — Python = 骨格 / Markdown = 知能

17 / ズームイン① — メインループ(agent.py)

```
while True:
    iteration += 1
    if max_iterations and iteration > max_iterations: break
    client = create_client(project_dir, model) # 毎回 new client
    prompt = initializer_prompt if first_run else coding_prompt
    async with client:
        await run_agent_session(client, prompt, project_dir)
    await asyncio.sleep(3)
```

ハーネスのメイン責務 4 つ

①

コンテキスト窓を真っさらに

②

プロンプトを切り替え

③

ファイルシステムに状態を保持

④

次セッション待機

18 / ズームイン② — 知能はプロンプトに(coding_prompt.md)

```
## STEP 1: GET YOUR BEARINGS (MANDATORY)
```

```
1. pwd  
2. ls -la  
3. cat app_spec.txt  
4. cat feature_list.json | head -50  
5. cat claude-progress.txt  
6. git log --oneline -20  
7. cat feature_list.json | grep '"passes": false' | wc -l
```

→ コンテキストを失った人間が現場復帰する時の行動様式そのもの

feature_list が JSON である理由

= Markdown だと Claude が勝手に編集する。passes フィールドを true にする更新だけ許可。
プロンプト設計が『信頼の設計』になっている

危険なツール呼び出しは事前に止める

事業視点 : = 本番 DB の事故・誤削除を技術的にゼロにする

今すぐやれること ~/.claude/settings.json に追加(コピペ可)

```
{ "hooks": { "PreToolUse": [{  
  "matcher": "Bash",  
  "hooks": [{  
    "type": "command",  
    "command": "jq -r '.tool_input.command // \\\"\\\"' | grep -qE 'rm -rf|DROP TABLE' && exit 2 || exit 0"  
  }]  
}] }
```

正規表現マッチは入門・デモ用。

本番では if フィールドや MCP ベースの構造化判定、LLM judge ハンドラを使う

rm -rf は最初の 1 個。次は production/、.env など自分のリポで本当に守りたいパス

jq が無ければ brew/apt install jq

何が起きたかを後から再現できる

事業視点: = AI が行った操作を後追いできる = 監査・コンプラ要件への布石。AI に任せられる根拠を残す

今すぐやれること *PostToolUse* で全 *Bash* 実行を *audit.log* に追記

```
{ "hooks": { "PostToolUse": [{  
  "matcher": "Bash",  
  "hooks": [{  
    "type": "command",  
    "command": "jq -r '[now|todate, .tool_input.command] | @tsv' >> ~/.claude/audit.log"  
  }]  
}] }
```

金融・医療・公共などデータの取扱に厳しい業界では、これが監査要件

事業責任者: 自社で hook の audit ログがあるか聞いてみる

1 つ落ちても全体は死なない設計

"If the container died, the harness caught the failure as a tool-call error and passed it back to Claude."
ペットではなく家畜 (cattle-not-pets) — 業界一般の比喻

★ **Stop hook 1 個で潰せる失敗**: ハーネスがない時の 3 失敗 (スライド 8) — ①一発書き / ②E2E 未確認 / ③早期完了申告
→ **Stop hook で ①と③ を同時に潰す**

事業視点: = AI が "終わった" と言っても、テスト緑じゃないと終われないルールを技術的に強制

今すぐやれること: Stop hook でテスト走らせ、落ちたら exit 2

```
{ "hooks": { "Stop": [ { "hooks": [ {  
  "type": "command",  
  "command": "${TEST_CMD:-npm test} || exit 2"  
} ] ] ] }
```

言語別例: pytest / go test / cargo test / rspec

AI を信頼する前に、信頼できる仕組みを作る

22a / 原則④ — 足し算ではなく引き算(剥がす運用)

原則 ①～③ は hook を『書く側』の指針 / 原則 ④ は hook を『剥がす側』の指針
— レイヤが違う

「ハーネスは "モデル単独でできないことの仮定" を符号化したもの。モデルが進化するとその仮定は陳腐化する」

"Harnesses encode assumptions that go stale as models improve." — Managed Agents, 2026/4

実例: Opus 4.6 リリースで autonomous-coding の sprint 構造を削除しても性能維持

事業視点: = 一度入れたガードを"いつ外すか"を運用に組み込む。技術的負債と同様、ハーネスも放置すると負債化する

今すぐやれること(カレンダー登録のみ): 半年後に自分の hooks 一覧を読み直し、もう要らないものを消す

ハーネスは育てるものではなく、剥がしていくもの

想定される疑問： hooks を勉強しても、Claude がもっと賢くなったら全部要らなくなるのでは ？
→ 答えは半分 Yes、半分 No

「モデルが賢くなるほど、モデル単独ではできない "より複雑なタスク" に挑むハーネスを作る余地が広がる」

"the better the models get, the more space there is to develop harnesses..." — Harness design, 2026/3

個別の hook は陳腐化する
でも、"外側で支える" 設計スキル自体は陳腐化しない
→ だから今日学ぶ価値がある

剥がす"目"は持ち続ける —— これが今日 1 番伝えたいこと

23 / 行動喚起 — 今すぐ → 今週 → 今月の 3 段ロードマップ

今すぐ 5 分

- ・rm -rf ブロックを 1 個コピペ
- ・claude --debug で動作確認

今週中

- ・SessionStart で git 状態注入を追加
- ・動いた hook を #ai-shakai に報告
- ・次回、面白い hook を紹介

今月中

- ・.claude/settings.json に移して git commit
- ・個人 hook → チーム hook に昇格
- ・アンチパターン 3 つを読み合わせ

役割別 1 行(軸: 自分の役割としての可視化責任)

エンジニア個人

自分の事故を可視化 (hook で記録)

チームリード

チーム標準を可視化 (settings.json を git に)

事業責任者

チームの取り組みを可視化 (hook の状況を聞く)

QR コード → コピペ用 settings.json (GitHub Gist)

ハーネス＝モデルへの『信頼の前提条件』

AI が暴走しない仕組みを、人が用意する。

プロンプトでお願いするのではなく、ルールとして決定論的に保証する。

質問・感想はチャット欄 / X / 勉強会チャンネルへ。ありがとうございました